

Project Objectives:

- To provide a hands-on guide to disassembling and analyzing malware with Ghidra.
- To enable readers to recognize common malware functionalities, such as persistence mechanisms, anti-analysis techniques, and network communications.
- To equip users with the skills to interpret low-level assembly code and understand high-level behaviours.

GitHub Repository Components

A GitHub repository for this project could include:

1. README.md
 - Overview: Brief description of Ghidra, its installation, and how it is used for malware analysis.
 - Requirements: List of requirements, including Ghidra, a VM setup, and any sample binaries (preferably benign samples or detailed steps to safely acquire controlled malware samples).
 - Project Outline: A breakdown of the tutorial steps, linking to various sections or files within the repository.
2. Tutorial Steps (Markdown or Jupyter Notebooks)
 - Step-by-step instructions, broken down into digestible sections, such as:
 - ✦ Environment Setup: Instructions on setting up a virtualized and isolated environment.
 - ✦ Initial Analysis: How to load a binary into Ghidra, perform auto-analysis, and identify entry points.
 - ✦ Function Analysis: Methods for identifying and analyzing critical functions, decompiling code, and using Ghidra's cross-referencing features.
 - ✦ String and Import Analysis: How to locate and interpret strings and imports related to malicious behavior.
 - ✦ Advanced Techniques: Dynamic analysis, control flow graphs, custom scripting with Ghidra's Python or Java capabilities, and obfuscation detection.

3 Example Scripts (Python or Java)

- Scripts to automate repetitive analysis tasks, such as:
 - ✦ Extracting and listing all string references.
 - ✦ Identifying network-related functions or common indicators of malware.
 - ✦ Labeling functions and data segments based on known patterns (e.g., file system manipulation, registry edits).
- Script Examples: Python or Java code snippets demonstrating API usage in Ghidra (using Ghidra's Headless Analyzer, for example, for automation).

4 Sample Data (Benign Binary or Hex Dump) ○ Due to safety and ethical concerns, include only safe, benign binaries or a hex dump example rather than live malware. This can allow users to learn and practice without the risk of executing malicious code.

- Sample binaries designed to exhibit typical malware-like behaviors, such as:
 - ✦ Executing system calls.
 - ✦ Writing to files.
 - ✦ Sending network data.
- Alternatively, a dummy binary generator script that creates binaries with similar structures to malware but without harmful payloads.

5 Documentation and Comments

- Include explanations of the code within each script, detailing what each function does.
- Inline comments in disassembly views, providing readers with insights into assembly commands, control flow structures, and the decompiled code.

6 Report Template ○ A reporting template for documenting findings, which could be in Markdown or PDF format. The template should cover:

- ✦ Summary of Analysis: Brief description of the suspected malware.
- ✦ Function Analysis: Summary of the key functions, their purpose, and the findings.
- ✦ Behavioral Indicators: Summary of malware behaviors, such as persistence, file system manipulation, or network communications.
- ✦ Recommendations: Suggestions on further actions, such as containment, eradication, or further analysis.

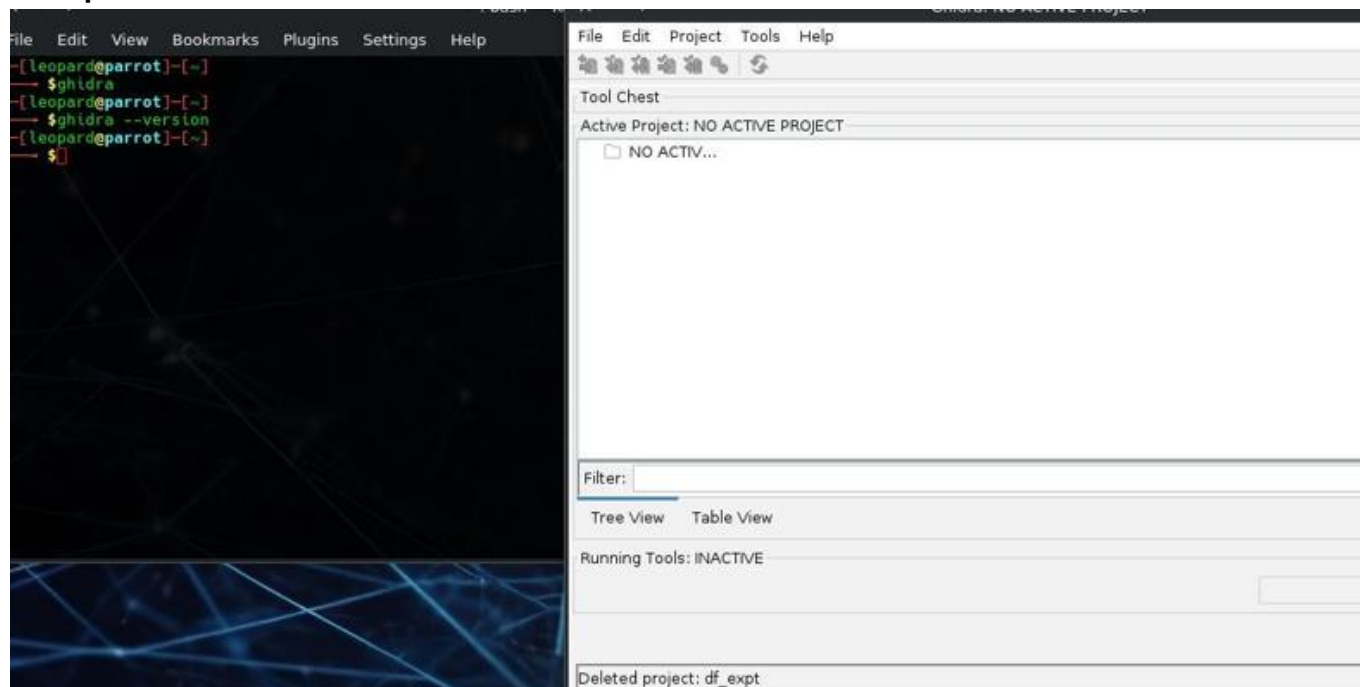
Example Repository Structure plaintext ghidra-
malware-analysis-project/

```
|— README.md
|— docs/
|   |— Tutorial_Step1.md
|   |— Tutorial_Step2.md
|   |— Tutorial_StepN.md
|— scripts/
|   |— extract_strings.py
|   |— network_analysis.py
|   |— label_functions.py
|— samples/
|   |— benign_sample1.bin
|   |— benign_sample2.hex
|— templates/
|   |— analysis_report_template.md
|— screenshots/
|   |— function_graph_example.png
|   |— decompiler_example.png
```

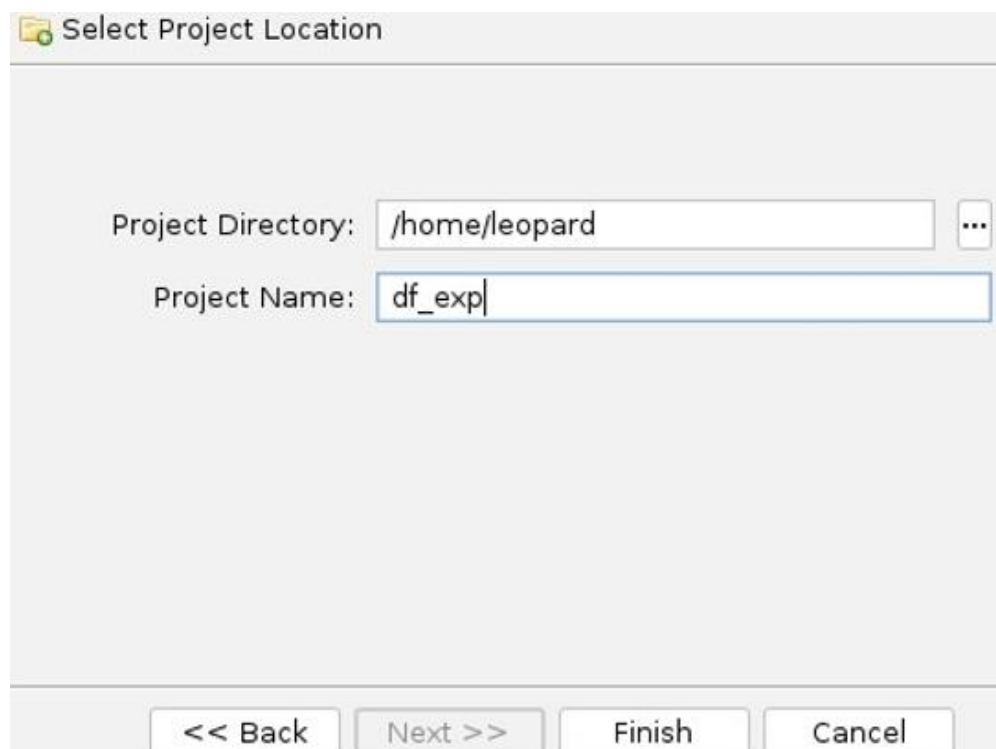
Project Development and Future Work

- Expand Scripts: Continue adding Ghidra scripts to automate complex analysis tasks.
- Community Contributions: Encourage community members to contribute additional benign samples, analysis scripts, and documentation improvements.
- Advanced Topics: Include guides for advanced techniques, such as unpacking obfuscated code, identifying encryption routines, and using the P-Code emulator for runtime analysis.

Step 1: Ghidra



Step 2: Creating the project



Step 3: Importing the .exe file

File Edit Project Tools Help

Active Project: df_exp

df_exp

X ^ v Import /home/leopard/Downloa...r-windows-x86_64-15.0.23.exe

Format: Portable Executable (PE)
Language: x86:LE:64:default:clangwindows
Destination Folder: /
Program Name: mullvad-browser-windows-x86_64-15.0.23.exe
☐ Mirror Filesystem Options...

OK Cancel

Filter:
Tree View Table View

```

Project File Name: mullvad-browser-windows-x86_64-15.0.23.exe
Last Modified: Mon Sep 21 23:07:23 IST 2026
Readonly: false
Program Name: mullvad-browser-windows-x86_64-15.0.23.exe
Language ID: x86:LE:64:default (4.6)
Compiler ID: clangwindows
Processor: x86
Endian: Little
Address Size: 64
Minimum Address: 140000000
Maximum Address: 1400acfff
# of Bytes: 694640
# of Memory Blocks: 7
# of Instructions: 0
# of Defined Data: 1038
# of Functions: 79
# of Symbols: 293
# of Data Types: 49
# of Data Type Categories: 3
Compiler: clang:unknown
Created With Ghidra Version: 12.0.4
Date Created: Mon Sep 21 23:07:22 IST 2026
Executable Format: Portable Executable (PE)
Executable Location: /home/leopard/Downloads/mullvad-browser-windows-x86_64-15.0.23.exe
Executable MD5: 3e024598db165e664713b46a73730e16
Executable SHA256: 46087a4cf549a411a578e9bcc9ab8481a2084d7a9b953ce57407e011d78e73b2
FSRL: file:///home/leopard/Downloads/mullvad-browser-windows-x86_64-15.0.23.e
PE Property[FileDescription]: Mullvad Browser Installer
PE Property[FileVersion]: 15.0.23
PE Property[LegalCopyright]: © 2026 Mullvad, Tor Browser and Mozilla Developers
PE Property[ProductName]: Mullvad Browser
PE Property[ProductVersion]: 15.0.23
PE Property[Translation]: 4b00409
Preferred Root Namespace Category:
Relocatable: false
SectionAlignment: 4096

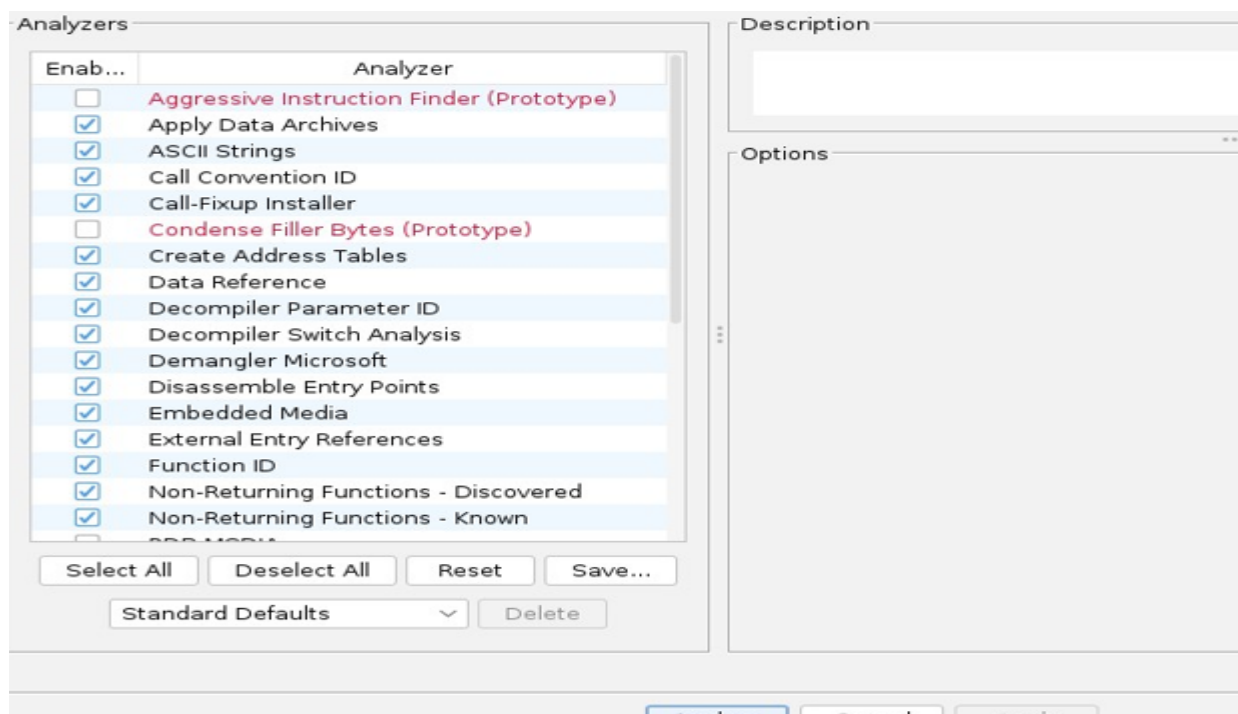
```

Additional Information
Loading file:///home/leopard/Downloads/mullvad-browser-windows-x86_64-15.0.23.exe?MD5=3e024598db165e6647

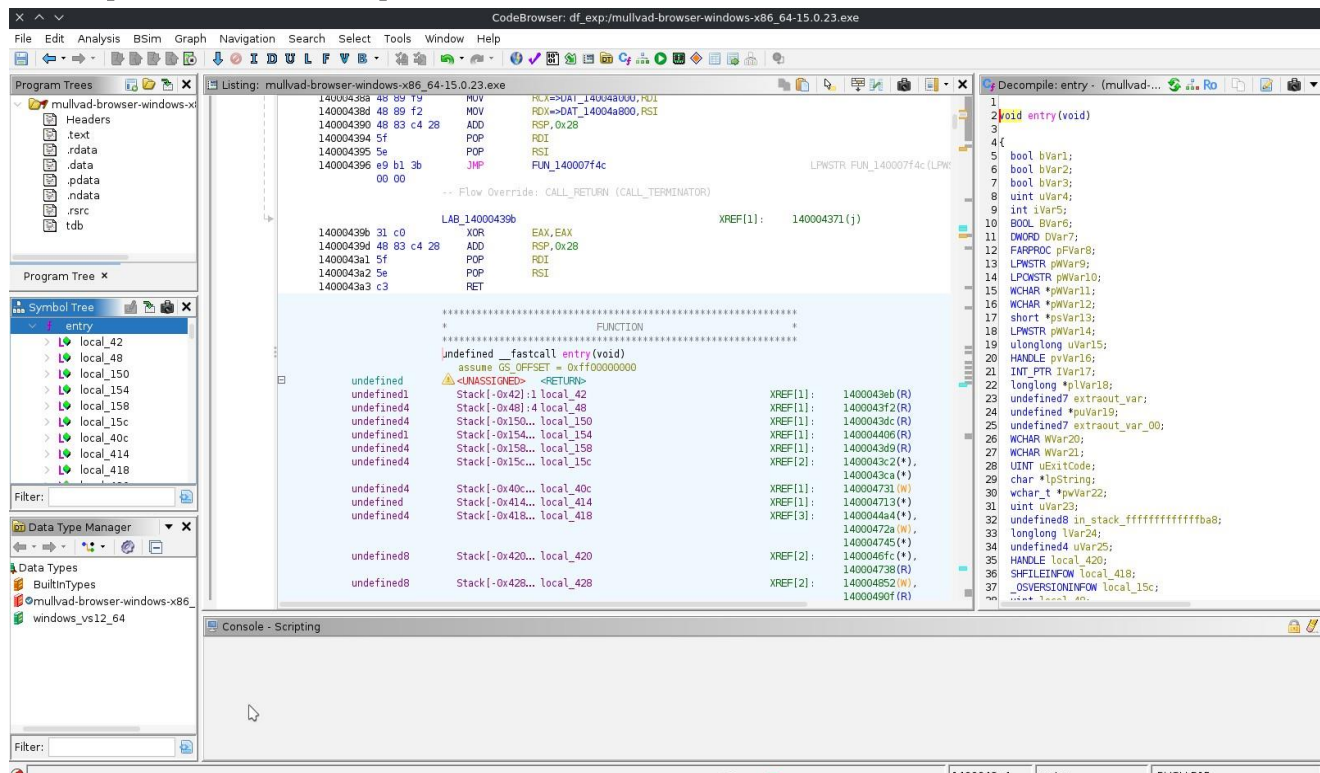
Searching 9 paths for library ADVAPI32.DLL...
Library not found.

Searching 9 paths for library COMCTL32.DLL...
Library not found.

Step 4: Analyzing the file:



Step 5: Decompiler:



Step 6: Strings

CodeBrowser: df_exp/multivad-browser-windows-x86_64-15.0.23.exe

Listing: multivad-browser-windows-x86_64-15.0.23.exe

Defined Strings - 248 items

Location	String Value	String Repre...	Data Type
140000000	MZ	"MZ"	char[2]
140000078	PE	"PE"	char[4]
140000180	.text	".text"	char[8]
1400001a8	.rdata	".rdata"	char[8]
1400001d0	.data	".data"	char[8]
1400001f8	.pdata	".pdata"	char[8]
140000220	.ndata	".ndata"	char[8]
140000248	.rsrc	".rsrc"	char[8]
14000b1e0	unpacking da...	"unpacking da..."	unicode
14000b20a	verifying inst...	"verifying inst..."	unicode
14000b23e	...%d%%	"u"...%d%%"	unicode
14000b250	Error launchi...	"Error launc..."	unicode
14000b284	installer integ...	"u"installer int..."	unicode
14000b41c	Error writing t...	"u>Error writin..."	unicode
14000b4b0	Error writing t...	"u>Error writin..."	unicode
14000b548	UXTHEME	"UXTHEME"	ds
14000b548	USERENV	"USERENV"	ds
14000b550	SETUPAPI	"SETUPAPI"	ds
14000b559	APPHelp	"APPHelp"	ds
14000b561	PROPSYS	"PROPSYS"	ds
14000b570	CRYPTBASE	"CRYPTBASE"	ds
14000b57a	OLEACC	"OLEACC"	ds
14000b589	NTMARTA	"NTMARTA"	ds
14000b594	NSIS Error	"u"NSIS Error"	unicode
14000b5aa	Temp	"u"Temp"	unicode
14000b5be	TEMP	"u"TEMP"	unicode

Step 7: Functions

CodeBrowser: df_exp/multivad-browser-windows-x86_64-15.0.23.exe

Listing: multivad-browser-windows-x86_64-15.0.23.exe

entry

```

14000438a 48 89 f9 MOV RCX, RAX
14000438d 48 89 f2 MOV RDX, RAX
140004390 48 83 c4 28 ADD RSP, 0x28
140004394 5f POP RDI
140004395 5e POP RSI
140004396 e9 b1 3b JMP FUN_140007f4c
00 00

-- Flow Override: CALL_RETURN (CALL_TERMINATOR)

14000439b 31 c0 XOR EAX, EAX
14000439d 48 83 c4 28 ADD RSP, 0x28
1400043a1 5f POP RDI
1400043a2 5e POP RSI
1400043a3 c3 RET
  
```

Stack frame setup:

```

Stack[-0x42]: 1 local_42
Stack[-0x48]: 4 local_48
Stack[-0x150]: local_150
Stack[-0x154]: local_154
Stack[-0x158]: local_158
Stack[-0x15c]: local_15c
Stack[-0x40c]: local_40c
Stack[-0x414]: local_414
Stack[-0x418]: local_418
Stack[-0x420]: local_420
Stack[-0x428]: local_428
  
```

Function call:

```

FUN_140007f4c (LPWSTR)
  
```

8:Exporting the results:

```
Project File Name: | mullvad-browser-windows-x86_64-15.0.23.exe
Last Modified:      Mon Sep 21 23:29:40 IST 2026
Readonly:           false
Program Name:       mullvad-browser-windows-x86_64-15.0.23.exe
Language ID:        x86:LE:64:default (4.6)
Compiler ID:        clangwindows
Processor:          x86
Endian:             Little
Address Size:       64
Minimum Address:    140000000
Maximum Address:    ff0000184f
# of Bytes:         700864
# of Memory Blocks: 8
# of Instructions:   9362
# of Defined Data:   1423
# of Functions:      254
# of Symbols:        1367
# of Data Types:     311
# of Data Type Categories: 22
Analyzed:           true
Compiler:           clang:unknown
Created With Ghidra Version: 12.0.4
Date Created:       Mon Sep 21 23:07:22 IST 2026
Executable Format:   Portable Executable (PE)
Executable Location: /home/leopard/Downloads/mullvad-browser-windows-x86_64-15.0.23.exe
Executable MD5:      3e024598db165e664713b46a73730e16
Executable SHA256:   46087a4cf549a411a578e9bcc9ab8481a2084d7a9b953ce57407e011d78e73b2
FSRL:               file:///home/leopard/Downloads/mullvad-browser-windows-x86_64-15.0.23.
PE Property[FileDescription]: Mullvad Browser Installer
PE Property[FileVersion]:    15.0.23
PE Property[LegalCopyright]: © 2026 Mullvad, Tor Browser and Mozilla Developers
PE Property[ProductName]:    Mullvad Browser
PE Property[ProductVersion]: 15.0.23
PE Property[Translation]:    4b00409
Preferred Root Namespace Category:
RTTI Found:           false
Relocatable:          false
SectionAlignment:     4096
Should Ask To Analyze: false
```

Additional Information

```
Destination file:      /home/leopard/mullvad-browser-windows-x86_64-15.0.23.exe.gzf
Destination file Size: 97594647
Format:                Ghidra Zip File
```

Rubrics

Criteria&Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity&Submission Regularity	3	
2. Application of Forensic Tools&Practical Execution	3	
3. Documentation&Reporting	2	
4. Engagement, Problem-Solving&Team Collaboration	2	
Total	10	

Result: The experiment successfully disassembled and statically analyzed the executable binary using the-Ghidra reverse engineering framework. By running Ghidra's auto-analysis modules, the investigator successfully navigated the low-level assembly control flow, isolated critical functions via cross-references, and extracted readable strings and code imports to securely map out malware-like behaviours without executing the malicious payload.

